

Lecture 32. Overview of Iterative Methods

With this lecture the flavor of the book changes. We move from direct methods, a classical topic that is rather thoroughly understood, to the relatively untamed territory of iterative methods. These are the methods that seem likely to dominate the large-scale computations of the future.

Why Iterate?

The importance of iterative algorithms in linear algebra stems from a simple fact: noniterative or “direct” algorithms for general matrices require $O(m^3)$ work. This is too much! It is too much both in the absolute sense that m^3 is huge when m is large, and in the relative sense that since the input to most matrix problems involves only $O(m^2)$ numbers, it seems unreasonable that $O(m^3)$ work must be expended in solving them.

The following table gives a thumbnail history of matrix computations over the years:

1950: $m = 20$	(Wilkinson)
1965: $m = 200$	(Forsythe and Moler)
1980: $m = 2000$	(LINPACK)
1995: $m = 20000$	(LAPACK)

These numbers represent a rough approximation to what dimensions might have been considered “very large” for a dense, direct matrix computation at the indicated dates. In the “Forsythe and Moler era” of the mid-1960s (named here after an influential textbook published in 1967), for example, a matrix of

dimension in the hundreds was large, stretching the limits of what could be calculated on available machines in a reasonable amount of time.

Evidently, in the course of forty-five years, the dimensions of tractable matrix problems have increased by a factor of 10^3 . This progress is impressive, but it pales beside the progress achieved by computer hardware in the same period—a speedup by a factor of 10^9 , from flops to gigaflops. In the fact that 10^9 is the cube of 10^3 , we see played out in history the $O(m^3)$ bottleneck of direct matrix algorithms.

To put it another way, if matrix problems could be solved in $O(m^2)$ instead of $O(m^3)$ operations, some of the matrices being treated today might be 10 to 100 times larger. This is the aim, achieved for some matrices but not others, of matrix iterative methods.

Structure, Sparsity, and Black Boxes

Of course, it is not at all obvious that the $O(m^3)$ bottleneck can be beaten, and indeed, for “random” matrix problems, very likely it cannot. However, the large matrix problems that arise in practice are far from random, and there is a simple reason for this. Small matrices, say with dimension 3 or 30, may arise directly with more or less arbitrary entries in scientific problems—as representations of the relations between three forces in a structure, perhaps, or between thirty species in a chemical reaction. Large matrices, by contrast, usually arise indirectly in the discretization of differential or integral equations. One might say that if m is very large, it is probably an approximation to ∞ . It follows that most large matrices of computational interest are simpler than their vast numbers of individual entries might suggest. They have some kind of structure, and as the years have gone by, ways have been found to exploit this structure in more and more contexts.

The most obvious structure that a large matrix may have is *sparsity*, i.e., preponderance of zero entries. (The opposite of *sparse* is *dense*.) For example, a finite difference discretization of a partial differential equation may lead to a matrix of dimension $m = 10^5$ with only $\nu = 10$ nonzero entries per row. This kind of structure is readily exploited by the iterative methods we shall discuss, for these algorithms use a matrix in the form of a *black box*:

$$x \longrightarrow \boxed{\begin{array}{c} \text{BLACK} \\ \text{BOX} \end{array}} \longrightarrow Ax.$$

The iterative algorithm requires nothing more than the ability to determine Ax for any x , which in a computer program will be effected by a procedure whose internal workings need be of no concern to the designer of the iterative algorithm. (Some iterative algorithms also require the computation of A^*x .) For the example of a sparse matrix A , it is easy to design a procedure to compute Ax in only $O(\nu m)$ rather than $O(m^2)$ operations. This is in marked

contrast to the algorithms of direct linear algebra, such as Gaussian or Householder triangularization, which explicitly manipulate matrix entries so as to introduce zeros, but in the process generally destroy sparsity.

Historically, sparsity has been the kind of structure exploited most often in iterative matrix computations. (Sparsity is also exploited by fast direct methods such as nested dissection or minimal degree reordering, not discussed in this book.) More recently, it has become apparent that other kinds of matrix structure may also be exploitable, even though the matrices involved are dense. For example, the solution of integral equations by numerical methods typically leads to dense matrix problems; in engineering these are called boundary element or panel methods. The coefficients of such matrices often have a great deal of regularity in them, and the quest for ways to take advantage of this regularity, for example by multipole methods or wavelet expansions, is an active area of research today. The black boxes involved in implementing such methods may have thousands of lines of code in them and be based on ideas that only specialists understand.

Projection into Krylov Subspaces

The iterative methods that occupy the remainder of this book are based on the idea of projecting an m -dimensional problem into a lower-dimensional Krylov subspace. Given a matrix A and a vector b , the associated *Krylov sequence* is the set of vectors $b, Ab, A^2b, A^3b, \dots$, which can be computed by the black box in the form $b, Ab, A(Ab), A(A(Ab)), \dots$. The corresponding *Krylov subspaces* are the spaces spanned by successively larger groups of these vectors.

Specifically, the algorithms that we shall discuss can be arranged in the following table:

	$Ax = b$	$Ax = \lambda x$
$A = A^*$	CG	Lanczos
$A \neq A^*$	GMRES CGN BCG et al.	Arnoldi

(This field is full of acronyms! CG, for example, stands for conjugate gradients, and, by the way, requires that A be positive definite as well as hermitian.) In each of these methods, the result of projection into the Krylov subspaces is that the original matrix problem is reduced to a sequence of matrix problems of dimensions $n = 1, 2, 3, \dots$. When A is hermitian, the reduced matrices are tridiagonal, whereas in the nonhermitian case they have Hessenberg form.

Thus the Arnoldi iteration, for example, approximates eigenvalues of a large matrix by computing eigenvalues of certain Hessenberg matrices of successively larger dimensions.

Number of Steps, Work per Step, and Preconditioning

Gaussian elimination, QR factorization, and most other algorithms of dense linear algebra fit the following pattern: there are $O(m)$ steps, each requiring $O(m^2)$ work, for a total work estimate of $O(m^3)$. (Of course these figures, especially the second, may change on a parallel computer.) For iterative methods, the same figures still apply, but now they represent a typical worst-case behavior. When these methods succeed, they may do so by reducing one or both of these factors.

We shall see that the number of steps required for convergence to a satisfactory precision typically depends on spectral properties of the matrix A , if the word “spectral” is interpreted broadly. For example, the conjugate gradient iteration is guaranteed to solve a hermitian positive definite system $Ax = b$ quickly if the eigenvalues of A are clustered well away from the origin. Similarly, the Lanczos iteration is guaranteed to compute certain eigenvalues of a real hermitian matrix quickly if those eigenvalues are well separated from the rest of the spectrum (and if the initial vector that starts the iteration is suitably generic). The analysis of the convergence rates of these methods is a fascinating study that depends on the mathematical field of approximation theory. Specifically, the convergence of Krylov subspace iterative algorithms is closely related to problems of approximation of functions $f(z)$ by polynomials $p(z)$ on subsets of the real axis or the complex plane.

The work per step in a matrix iteration depends mainly on the structure of the matrix and on what advantage is taken of this structure in the $x \mapsto Ax$ black box.

The ideal iterative method in linear algebra reduces the number of steps from m to $O(1)$ and the work per step from $O(m^2)$ to $O(m)$, reducing the total work from $O(m^3)$ to $O(m)$. Such extraordinary speedups do occur in practical problems, but a more typical improvement is perhaps from $O(m^3)$ to $O(m^2)$. In a practical large-scale engineering computation of the mid-1990s, where iterative algorithms are successful, perhaps a typical result is that they beat direct algorithms by a factor on the order of 10. As machines get faster and m gets larger in the future, this factor will increase and iterative algorithms will become more important, illustrating the fundamental law of computer science: the faster the computer, the greater the importance of speed of algorithms.

Exact vs. Approximate Solutions

Matrix iterative methods are approximate in the sense that in principle they do not deliver exact answers, even in the absence of rounding errors, at least

when carried to the number of iterative steps that is of practical interest. This property tends to make newcomers to these ideas uneasy; they may feel that iteration is an “engineering solution” of little elegance and doubtful reliability. This uneasiness tends to diminish as one gets to know these methods better. After all, even direct methods are inexact when carried out on a computer: one hopes for answers accurate to machine precision, no better. Since iterative methods too may be used to achieve the full accuracy of machine precision, the fact that they are in principle approximate need have little significance. As for elegance, the ideas that arise here are some of the most beautiful in numerical linear algebra.

These points are illustrated in Figure 32.1.

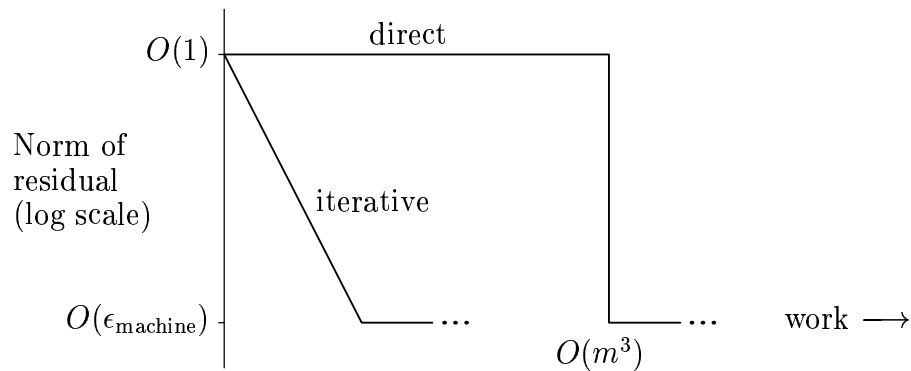


Figure 32.1. *Schematic illustration of convergence of direct and iterative methods. Under favorable circumstances, the iterative method converges geometrically until the residual is on the order of $\epsilon_{\text{machine}}$. The direct method makes no progress at all until $O(m^3)$ operations are completed, at which point the residual is again on the order of $\epsilon_{\text{machine}}$.*

Direct Methods That Beat $O(m^3)$

Finally, we must mention that there exist direct algorithms—finite, in principle exact—that solve $Ax = b$ and related problems for dense matrices in less than $O(m^3)$ operations. The first algorithm of this kind was discovered in 1969 by Volker Strassen, who reduced Gauss’s exponent of 3 to $\log_2(7) \approx 2.81$, and subsequent improvements have reduced the best known exponent to its current value of ≈ 2.376 due to Coppersmith and Winograd. The history of these best known exponents is recorded in Figure 32.2.

So far, these fast algorithms have had negligible impact on practical computation, for two reasons. One is that in general, little is known about their stability properties. More fundamental is the fact that although the exponents in the fast algorithms are striking, the crossover values of m at which

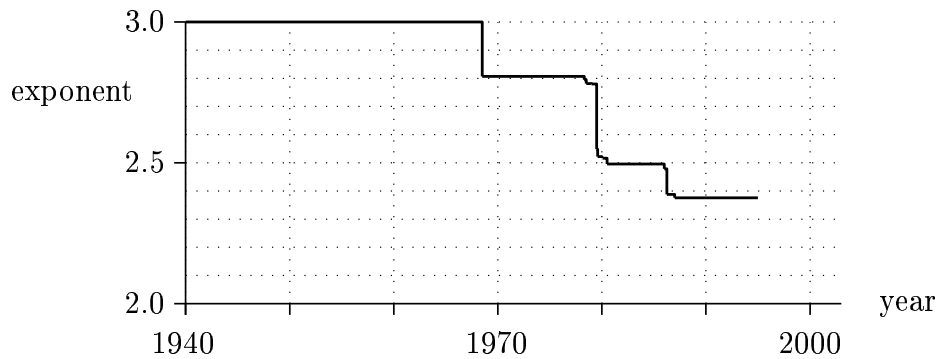


Figure 32.2. *Best known exponents for direct solution of $Ax = b$ (or equivalently, for computation of A^{-1} , AB , or $\det A$) for $m \times m$ matrices, as a function of time. Until 1968, the best known algorithms were of complexity $O(m^3)$. The currently best known algorithm solves $Ax = b$ in $O(m^{2.376})$ flops, but the constants are so large that this algorithm is impractical.*

they begin to beat standard methods are exceedingly high. Strassen's $m^{2.81}$ algorithm may be made to beat Gaussian elimination for values of m as low as 100, but because 2.81 is so close to 3, the victory never becomes dramatic for practical values of m . The existing methods with exponents much lower than this involve such large constant factors that they are slower than Gaussian elimination for the values of m attainable on current computers.

But what will happen in the future? The truth is that nobody knows. It is possible that tomorrow somebody will discover a “fast matrix inverse” that solves $Ax = b$ in $m^2 \log m$ floating point operations; or you, the reader, may do so this evening. Such a development would trigger the greatest upheaval in the history of numerical computation.

Exercises

32.1. An elliptic partial differential equation in three dimensions is discretized by a boundary element method. The result is a large dense linear system of equations in which each equation corresponds to a triangular surface element on a large sphere. To improve the accuracy, one must make the triangles smaller and thereby increase the number of equations, but the error shrinks only linearly in proportion to h , the diameter of the largest triangle.

A value of h is chosen, the system is solved by Gaussian elimination, and a solution accurate to two digits is obtained in one minute of computer time. It is decided that three digits of accuracy are needed. Assuming storage is

not a constraint, approximately how much time will be required for the new computation on the same computer?

32.2. Consider the block matrix product

$$\begin{bmatrix} W & X \\ Y & Z \end{bmatrix} = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix},$$

where, for simplicity, all the matrices $A, B, \dots, Y, Z$ are assumed to be square and of the same dimension.

(a) Given $A, B, \dots, G, H$, how many (i) matrix additions and (ii) matrix multiplications does it take to compute W, X, Y, Z by the obvious algorithm?

(b) Strassen showed that W, X, Y, Z can also be computed by the formulas

$$\begin{aligned} P_1 &= (A + D)(E + H), & P_5 &= (A + B)H, \\ P_2 &= (C + D)E, & P_6 &= (C - A)(E + F), \\ P_3 &= A(F - H), & P_7 &= (B - D)(G + H), \\ P_4 &= D(G - E), \\ W &= P_1 + P_4 - P_5 + P_7, & Y &= P_2 + P_4, \\ X &= P_3 + P_5, & Z &= P_1 + P_3 - P_2 + P_6. \end{aligned}$$

How many (i) matrix additions or subtractions and (ii) matrix multiplications are involved now?

(c) Show that by applying Strassen's formulas recursively, one can obtain an algorithm for multiplying matrices of dimension $m = 2^k$ with an operation count $O(m^{\log_2(7)})$ as $m \rightarrow \infty$.

(d) Write a recursive program that implements this idea, and give numerical evidence that your program works.